

Experiment ‘p_14_4_merge_sort_lexicographic_array’ Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 1

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 3

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 3

Problem Specification

Problem name: p_14_4_merge_sort_lexicographic_array

Natural language statement: null

Method signature: p_14_4_merge_sort_lexicographic_array(inputs: array<string>) returns (sorted: array<string>)

Ensures

- forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
- multiset(inputs[..]) == multiset(sorted[..])

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_14_4_merge_sort_lexicographic_array(inputs: array<string>) returns (sorted: array<
↪ string>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures
↪ multiset(inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPzCB20U6Q0wRRt5kE7Mug0ixGmw

Model: gpt-5.5-2026-04-23

Created at: 1780670334

Finish reason: stop

Usage: 316 tokens in, and 8894 tokens out

```
{
  var rest := inputs[..];
  var res: seq<string> := [];

  while |rest| > 0
    invariant multiset(res) + multiset(rest) == multiset(inputs[..])
    invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
    decreases |rest|
  {
    var oldRest := rest;
    assert multiset(res) + multiset(oldRest) == multiset(inputs[..]);

    var x := oldRest[0];
    rest := oldRest[1..];
    assert oldRest == [x] + rest;
    assert multiset(oldRest) == multiset([x]) + multiset(rest);

    var k := 0;
    while k < |res| && res[k] < x
```

```

invariant 0 <= k <= |res|
invariant forall t: int :: 0 <= t < k ==> res[t] < x
invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
decreases |res| - k
{
  k := k + 1;
}

assert forall t: int :: 0 <= t < k ==> res[t] <= x by {
  forall t: int | 0 <= t < k
    ensures res[t] <= x
  {
    assert res[t] < x;
  }
}

assert forall t: int :: k <= t < |res| ==> x <= res[t] by {
  forall t: int | k <= t < |res|
    ensures x <= res[t]
  {
    assert k < |res|;
    assert !(res[k] < x);
    assert x <= res[k];
    if t == k {
    } else {
      assert k < t;
      assert res[k] <= res[t];
    }
  }
}

var newRes := res[..k] + [x] + res[k..];
assert |newRes| == |res| + 1;

assert forall a: int, b: int :: 0 <= a < b < |newRes| ==> newRes[a] <= newRes[b] by {
  forall a: int, b: int | 0 <= a < b < |newRes|
    ensures newRes[a] <= newRes[b]
  {
    if b < k {
      assert newRes[a] == res[a];
      assert newRes[b] == res[b];
      assert res[a] <= res[b];
    } else if b == k {
      assert a < k;
      assert newRes[a] == res[a];
      assert newRes[b] == x;
      assert res[a] <= x;
    } else {
      assert k < b;
      assert b - 1 < |res|;
      assert newRes[b] == res[b - 1];
    }
  }
}

```

```

    if a < k {
      assert newRes[a] == res[a];
      assert a < b - 1;
      assert res[a] <= res[b - 1];
    } else if a == k {
      assert newRes[a] == x;
      assert k <= b - 1 < |res|;
      assert x <= res[b - 1];
    } else {
      assert k < a;
      assert newRes[a] == res[a - 1];
      assert 0 <= a - 1 < b - 1 < |res|;
      assert res[a - 1] <= res[b - 1];
    }
  }
}

assert res == res[..k] + res[k..];
assert multiset(newRes) == multiset(res) + multiset([x]);
assert multiset(newRes) + multiset(rest) == multiset(res) + multiset(oldRest);
assert multiset(newRes) + multiset(rest) == multiset(inputs[..]);

res := newRes;
}

assert |rest| == 0;
assert rest == [];
assert multiset(res) == multiset(inputs[..]);

sorted := new string[|res|](i => res[i]);
assert sorted.Length == |res|;
assert forall i: int :: 0 <= i < sorted.Length ==> sorted[i] == res[i];
assert sorted[..] == res;

assert forall i: int, j: int :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
  ↪ by {
    forall i: int, j: int | 0 <= i < j < sorted.Length
      ensures sorted[i] <= sorted[j]
    {
      assert sorted[i] == res[i];
      assert sorted[j] == res[j];
      assert res[i] <= res[j];
    }
  }

assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(45,8): Error: assertion
  ↪ might not hold
```

```
|
45 | assert x <= res[k];
|   ^^^^^
```

```
problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(104,38): Error: index out
  ↪ of range
```

```
|
104 | sorted := new string[|res|](i => res[i]);
|   ^
```

Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
↪ Dafny body, including the curly braces that surround it. Do not show the signature
↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQ2eQyPD1jqXaF69ODznuHZK2I6R

Model: gpt-5.5-2026-04-23

Created at: 1780670548

Finish reason: stop

Usage: 1723 tokens in, and 7223 tokens out

```
{
  var rest := inputs[..];
  var res: seq<string> := [];

  while |rest| > 0
    invariant multiset(res) + multiset(rest) == multiset(inputs[..])
    invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
    decreases |rest|
  {
    var oldRest := rest;
    assert multiset(res) + multiset(oldRest) == multiset(inputs[..]);

    var x := oldRest[0];
    rest := oldRest[1..];
    assert oldRest == [x] + rest;
    assert multiset(oldRest) == multiset([x]) + multiset(rest);

    var k := 0;
    while k < |res| && res[k] <= x
      invariant 0 <= k <= |res|
      invariant forall t: int :: 0 <= t < k ==> res[t] <= x
```

```

    invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
    decreases |res| - k
  {
    assert res[k] <= x;
    k := k + 1;
  }

  assert forall t: int :: k <= t < |res| ==> x <= res[t] by {
    forall t: int | k <= t < |res|
      ensures x <= res[t]
    {
      assert k < |res|;
      if res[k] <= x {
        assert k < |res| && res[k] <= x;
        assert false;
      }
      assert !(res[k] <= x);
      assert x <= res[k] || res[k] <= x;
      if !(x <= res[k]) {
        assert res[k] <= x;
        assert false;
      }
      assert x <= res[k];
      if t == k {
      } else {
        assert k < t;
        assert res[k] <= res[t];
      }
    }
  }

  var newRes := res[..k] + [x] + res[k..];
  assert |newRes| == |res| + 1;

  assert forall a: int, b: int :: 0 <= a < b < |newRes| ==> newRes[a] <= newRes[b] by {
    forall a: int, b: int | 0 <= a < b < |newRes|
      ensures newRes[a] <= newRes[b]
    {
      if b < k {
        assert newRes[a] == res[a];
        assert newRes[b] == res[b];
        assert res[a] <= res[b];
      } else if b == k {
        assert a < k;
        assert newRes[a] == res[a];
        assert newRes[b] == x;
        assert res[a] <= x;
      } else {
        assert k < b;
        assert b - 1 < |res|;
        assert newRes[b] == res[b - 1];
      }
    }
  }

```

```

    if a < k {
      assert newRes[a] == res[a];
      assert a < b - 1;
      assert res[a] <= res[b - 1];
    } else if a == k {
      assert newRes[a] == x;
      assert k <= b - 1 < |res|;
      assert x <= res[b - 1];
    } else {
      assert k < a;
      assert newRes[a] == res[a - 1];
      assert 0 <= a - 1 < b - 1 < |res|;
      assert res[a - 1] <= res[b - 1];
    }
  }
}

assert res == res[..k] + res[k..];
assert multiset(newRes) == multiset(res) + multiset([x]);
assert multiset(newRes) + multiset(rest) == multiset(res) + multiset(oldRest);
assert multiset(newRes) + multiset(rest) == multiset(inputs[..]);

res := newRes;
}

assert |rest| == 0;
assert rest == [];
assert multiset(res) == multiset(inputs[..]);

sorted := new string[|res|](i => if 0 <= i && i < |res| then res[i] else "");
assert sorted.Length == |res|;
assert forall i: int :: 0 <= i < sorted.Length ==> sorted[i] == res[i];
assert forall i: int :: 0 <= i < |res| ==> sorted[..][i] == res[i] by {
  forall i: int | 0 <= i < |res|
    ensures sorted[..][i] == res[i]
  {
    assert sorted[..][i] == sorted[i];
    assert sorted[i] == res[i];
  }
}
assert sorted[..] == res;

assert forall i: int, j: int :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
  ↪ by {
  forall i: int, j: int | 0 <= i < j < sorted.Length
    ensures sorted[i] <= sorted[j]
  {
    assert sorted[i] == res[i];
    assert sorted[j] == res[j];
    assert res[i] <= res[j];
  }
}

```

```

    }
  }

  assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(42,8): Error: assertion
  ↳ might not hold

```

```

|
42 | assert x <= res[k] || res[k] <= x;
|   ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQ5r9dlMa34K7uchXHUNU9LCUUz5

Model: gpt-5.5-2026-04-23

Created at: 1780670747

Finish reason: stop

Usage: 3122 tokens in, and 7605 tokens out

```

{
  var rest := inputs[..];
  var res: seq<string> := [];

  while |rest| > 0
    invariant multiset(res) + multiset(rest) == multiset(inputs[..])
    invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
    decreases |rest|
  {
    var oldRest := rest;
    assert multiset(res) + multiset(oldRest) == multiset(inputs[..]);

    var x := oldRest[0];
    rest := oldRest[1..];
    assert oldRest == [x] + rest;
    assert multiset(oldRest) == multiset([x]) + multiset(rest);

    var k := 0;
    while k < |res| && res[k] <= x

```



```

invariant 0 <= k <= |res|
invariant forall t: int :: 0 <= t < k ==> res[t] <= x
invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
decreases |res| - k
{
  assert res[k] <= x;
  k := k + 1;
}

if k < |res| {
  assert !(res[k] <= x);
  var y := res[k];
  var n := if |x| <= |y| then |x| else |y|;
  assert n <= |x| && n <= |y|;
  var p := 0;
  while p < n && x[p] == y[p]
    invariant 0 <= p <= n
    invariant n <= |x| && n <= |y|
    invariant forall q: int :: 0 <= q < p ==> x[q] == y[q]
    decreases n - p
  {
    p := p + 1;
  }
  if p == n {
    if |x| <= |y| {
      assert n == |x|;
      assert forall q: int :: 0 <= q < |x| ==> x[q] == y[q];
      assert x <= y;
    } else {
      assert n == |y|;
      assert forall q: int :: 0 <= q < |y| ==> y[q] == x[q];
      assert y <= x;
    }
  } else {
    assert p < n;
    assert p < |x| && p < |y|;
    assert x[p] != y[p];
    if x[p] < y[p] {
      assert x <= y;
    } else {
      assert y[p] < x[p];
      assert y <= x;
    }
  }
  assert x <= y || y <= x;
  assert y == res[k];
  assert x <= res[k] || res[k] <= x;
  assert x <= res[k];
}
assert k < |res| ==> x <= res[k];

```

```

assert forall t: int :: k <= t < |res| ==> x <= res[t] by {
  forall t: int | k <= t < |res|
    ensures x <= res[t]
  {
    assert k < |res|;
    assert x <= res[k];
    if t == k {
    } else {
      assert k < t;
      assert res[k] <= res[t];
    }
  }
}

var newRes := res[..k] + [x] + res[k..];
assert |newRes| == |res| + 1;

assert forall a: int, b: int :: 0 <= a < b < |newRes| ==> newRes[a] <= newRes[b] by {
  forall a: int, b: int | 0 <= a < b < |newRes|
    ensures newRes[a] <= newRes[b]
  {
    if b < k {
      assert newRes[a] == res[a];
      assert newRes[b] == res[b];
      assert res[a] <= res[b];
    } else if b == k {
      assert a < k;
      assert newRes[a] == res[a];
      assert newRes[b] == x;
      assert res[a] <= x;
    } else {
      assert k < b;
      assert b - 1 < |res|;
      assert newRes[b] == res[b - 1];
      if a < k {
        assert newRes[a] == res[a];
        assert a < b - 1;
        assert res[a] <= res[b - 1];
      } else if a == k {
        assert newRes[a] == x;
        assert k <= b - 1 < |res|;
        assert x <= res[b - 1];
      } else {
        assert k < a;
        assert newRes[a] == res[a - 1];
        assert 0 <= a - 1 < b - 1 < |res|;
        assert res[a - 1] <= res[b - 1];
      }
    }
  }
}

```

```

    assert res == res[..k] + res[k..];
    assert multiset(newRes) == multiset(res) + multiset([x]);
    assert multiset(newRes) + multiset(rest) == multiset(res) + multiset(oldRest);
    assert multiset(newRes) + multiset(rest) == multiset(inputs[..]);

    res := newRes;
}

assert |rest| == 0;
assert rest == [];
assert multiset(res) == multiset(inputs[..]);

sorted := new string[|res|](i => if 0 <= i && i < |res| then res[i] else "");
assert sorted.Length == |res|;
assert forall i: int :: 0 <= i < sorted.Length ==> sorted[i] == res[i];
assert forall i: int :: 0 <= i < |res| ==> sorted[..][i] == res[i] by {
    forall i: int | 0 <= i < |res|
        ensures sorted[..][i] == res[i]
    {
        assert sorted[..][i] == sorted[i];
        assert sorted[i] == res[i];
    }
}
assert sorted[..] == res;

assert forall i: int, j: int :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
    ↪ by {
    forall i: int, j: int | 0 <= i < j < sorted.Length
        ensures sorted[i] <= sorted[j]
    {
        assert sorted[i] == res[i];
        assert sorted[j] == res[j];
        assert res[i] <= res[j];
    }
}

assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(61,10): Error: assertion
    ↪ might not hold
|
61 | assert x <= y;
    | ~~~~~

```

```

problem_data/p_14_4_merge_sort_lexicographic_array/program.dfy(64,10): Error: assertion
  ↪ might not hold
|
64 | assert y <= x;
| ~~~~~

```

Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnQ9UXXjGPifhvkLsDi2BTrIqX5gS

Model: gpt-5.5-2026-04-23

Created at: 1780670972

Finish reason: length

Usage: 4965 tokens in, and 1117 tokens out

The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```

method p_14_4_merge_sort_lexicographic_array(inputs: array<string>) returns (sorted:
  ↪ array<string>)
  ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
  ensures multiset(inputs[..]) == multiset(sorted[..])
{
  var rest := inputs[..];
  var res: seq<string> := [];

  while |rest| > 0
    invariant multiset(res) + multiset(rest) == multiset(inputs[..])
    invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
    decreases |rest|
  {
    var oldRest := rest;
    assert multiset(res) + multiset(oldRest) == multiset(inputs[..]);

    var x := oldRest[0];
    rest := oldRest[1..];
    assert oldRest == [x] + rest;
    assert multiset(oldRest) == multiset([x]) + multiset(rest);

    var k := 0;
    while k < |res| && res[k] <= x

```

```

invariant 0 <= k <= |res|
invariant forall t: int :: 0 <= t < k ==> res[t] <= x
invariant forall i: int, j: int :: 0 <= i < j < |res| ==> res[i] <= res[j]
decreases |res| - k
{
  assert res[k] <= x;
  k := k + 1;
}

if k < |res| {
  assert !(res[k] <= x);
  var y := res[k];
  var n := if |x| <= |y| then |x| else |y|;
  assert n <= |x| && n <= |y|;
  var p := 0;
  while p < n && x[p] == y[p]
    invariant 0 <= p <= n
    invariant n <= |x| && n <= |y|
    invariant forall q: int :: 0 <= q < p ==> x[q] == y[q]
    decreases n - p
  {
    p := p + 1;
  }
  if p == n {
    if |x| <= |y| {
      assert n == |x|;
      assert forall q: int :: 0 <= q < |x| ==> x[q] == y[q];
      assert x <= y;
    } else {
      assert n == |y|;
      assert forall q: int :: 0 <= q < |y| ==> y[q] == x[q];
      assert y <= x;
    }
  } else {
    assert p < n;
    assert p < |x| && p < |y|;
    assert x[p] != y[p];
    if x[p] < y[p] {
      assert x <= y;
    } else {
      assert y[p] < x[p];
      assert y <= x;
    }
  }
  assert x <= y || y <= x;
  assert y == res[k];
  assert x <= res[k] || res[k] <= x;
  assert x <= res[k];
}
assert k < |res| ==> x <= res[k];

```

```

assert forall t: int :: k <= t < |res| ==> x <= res[t] by {
  forall t: int | k <= t < |res|
    ensures x <= res[t]
  {
    assert k < |res|;
    assert x <= res[k];
    if t == k {
    } else {
      assert k < t;
      assert res[k] <= res[t];
    }
  }
}

var newRes := res[..k] + [x] + res[k..];
assert |newRes| == |res| + 1;

assert forall a: int, b: int :: 0 <= a < b < |newRes| ==> newRes[a] <= newRes[b] by {
  forall a: int, b: int | 0 <= a < b < |newRes|
    ensures newRes[a] <= newRes[b]
  {
    if b < k {
      assert newRes[a] == res[a];
      assert newRes[b] == res[b];
      assert res[a] <= res[b];
    } else if b == k {
      assert a < k;
      assert newRes[a] == res[a];
      assert newRes[b] == x;
      assert res[a] <= x;
    } else {
      assert k < b;
      assert b - 1 < |res|;
      assert newRes[b] == res[b - 1];
      if a < k {
        assert newRes[a] == res[a];
        assert a < b - 1;
        assert res[a] <= res[b - 1];
      } else if a == k {
        assert newRes[a] == x;
        assert k <= b - 1 < |res|;
        assert x <= res[b - 1];
      } else {
        assert k < a;
        assert newRes[a] == res[a - 1];
        assert 0 <= a - 1 < b - 1 < |res|;
        assert res[a - 1] <= res[b - 1];
      }
    }
  }
}

```

```

    assert res == res[..k] + res[k..];
    assert multiset(newRes) == multiset(res) + multiset([x]);
    assert multiset(newRes) + multiset(rest) == multiset(res) + multiset(oldRest);
    assert multiset(newRes) + multiset(rest) == multiset(inputs[..]);

    res := newRes;
}

assert |rest| == 0;
assert rest == [];
assert multiset(res) == multiset(inputs[..]);

sorted := new string[|res|](i => if 0 <= i && i < |res| then res[i] else "");
assert sorted.Length == |res|;
assert forall i: int :: 0 <= i < sorted.Length ==> sorted[i] == res[i];
assert forall i: int :: 0 <= i < |res| ==> sorted[..][i] == res[i] by {
  forall i: int | 0 <= i < |res|
    ensures sorted[..][i] == res[i]
  {
    assert sorted[..][i] == sorted[i];
    assert sorted[i] == res[i];
  }
}
assert sorted[..] == res;

assert forall i: int, j: int :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
  ↪ by {
  forall i: int, j: int | 0 <= i < j < sorted.Length
    ensures sorted[i] <= sorted[j]
  {
    assert sorted[i] == res[i];
    assert sorted[j] == res[j];
    assert res[i] <= res[j];
  }
}

assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Total Token Usage

Input tokens: 10126

Output tokens: 24839

Reasoning tokens: 20505

Sum of 'total tokens': 34965

Experiment Timings

Overall Experiment started at 1780670333069, ended at 1780671004410, lasting 671341ms (671.34 seconds)

Iteration #4 started at 1780670969587, ended at 1780671004410, lasting 34823ms (34.82 seconds)

Iteration #1 started at 1780670333084, ended at 1780670546703, lasting 213619ms (213.62 seconds)

Iteration #2 started at 1780670546707, ended at 1780670742811, lasting 196104ms (196.10 seconds)

Iteration #3 started at 1780670742811, ended at 1780670969586, lasting 226775ms (226.78 seconds)